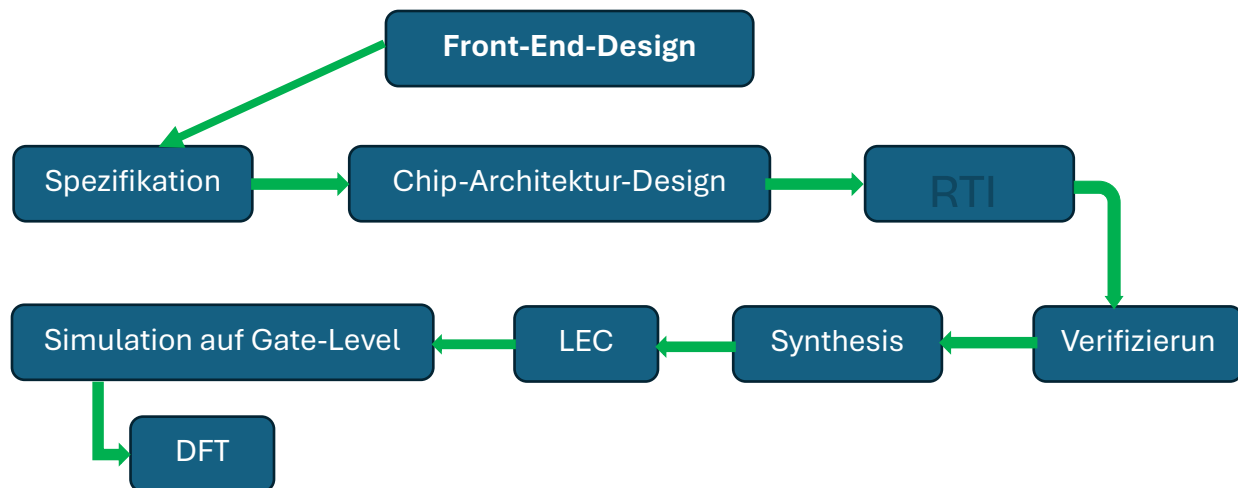




Spezifikation

z.B.

8-Bit Serial-in-Parallel Converter (SIPO)

Taktfrequenz > 1,5 GHz

Tech Prozess-Knoten: 130nm usw.

Chip-Architektur-Design

Dies ist für fortgeschrittenes Chip-Design. Für diese Übung ist es nicht relevant. Das Design von SIPO in einem allgemeinen Ansatz würde nicht ausreichen, um eine hohe Taktrate im 130nm-Prozess zu erreichen. In diesem Fall wird ein einfaches Multi-Clock-Domain- oder Pipelined-Design in Betracht gezogen.

RTL Design (HDL/Verilog)

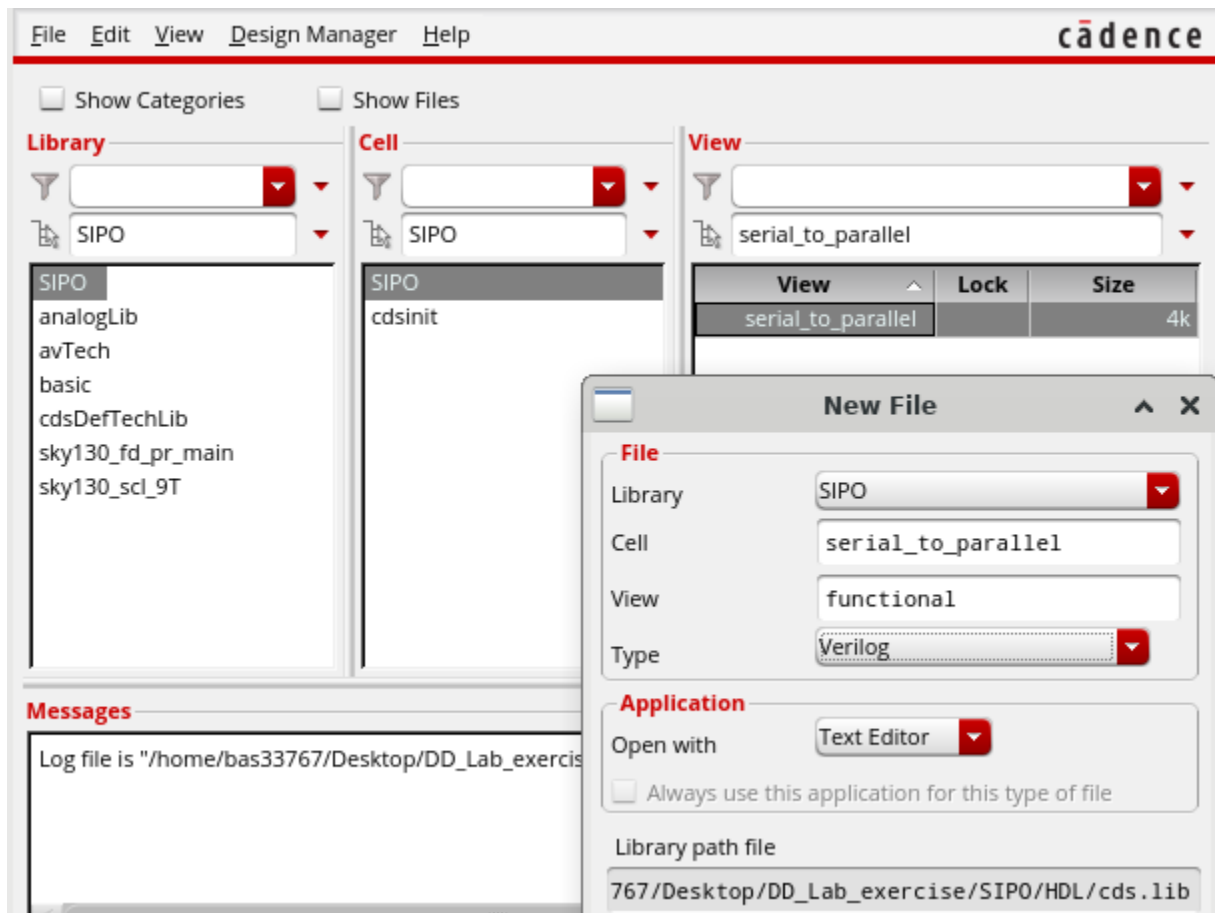
Design Entry (HDL → Verilog file)

Plain Text File:

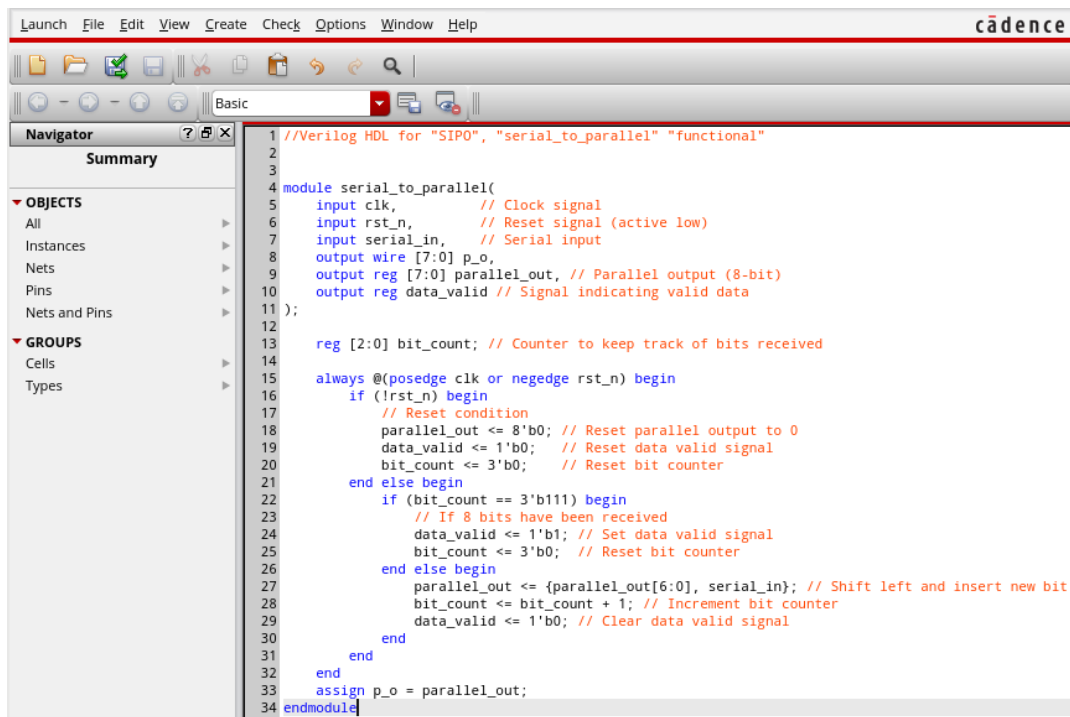
Die Eingabe des Entwurfs in Form einer HDL/Verilog-Datei kann in einer einfachen Textdatei mit der Erweiterung „*.v“ erfolgen. In diesem Fall kann die Kompilierung durch Aufrufen des Entwurfs in XCELIUM erfolgen.

In Virtuoso Studio:

Wie bei einem typischen Analogentwurf kann eine „**New cell view**“ über das Menü „**File**“ erstellt werden. Der Typ sollte „**Verilog**“ sein und mit dem „**Text Editor**“ geöffnet werden.

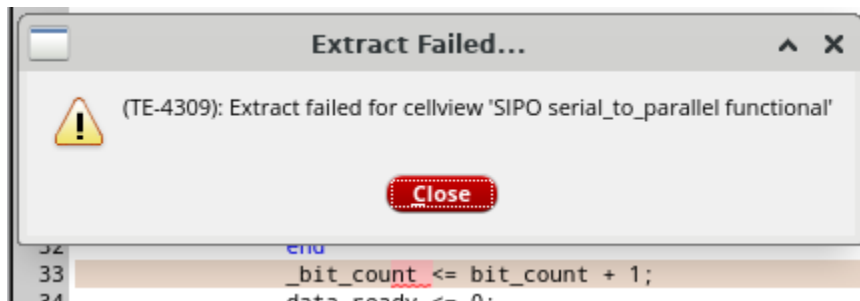


Der „Text Editor“ sollte wie folgt aussehen—



Hinweis: Der Code in der Abbildung wird nicht in der eigentlichen Simulation verwendet.

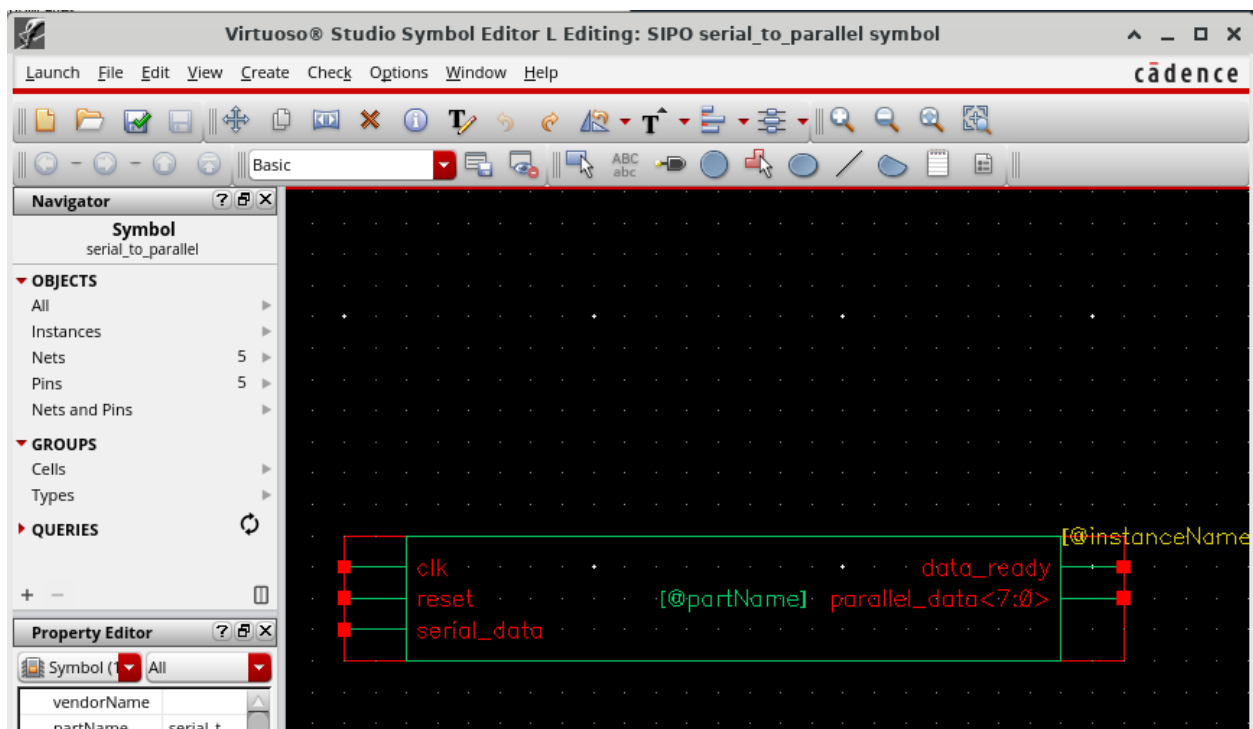
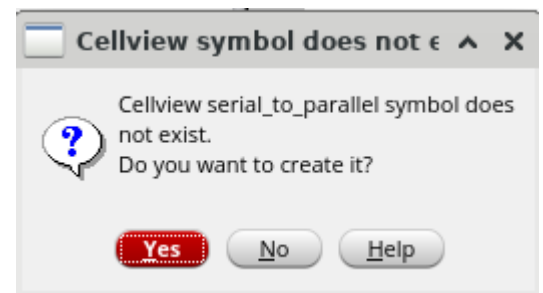
Der Entwurf kann mit der Schaltfläche „Build“ () erstellt und geprüft werden. Wenn es einen Syntaxfehler gibt, wird dieser wie folgt hervorgehoben—



Wenn alles in Ordnung ist, wird ein Symbol für den Entwurf erstellt (nur für neue „cellview“- falls noch kein Symbol erstellt wurde).

Es könnte für „Mixed Signal Design/Analyse“ nützlich sein, was nicht in den Rahmen dieser Übung fällt.

In ähnlicher Weise kann eine Testbench erstellt werden. Es ist jedoch nicht sinnvoll, eine Testbench in Virtuoso zu verwenden.



Verifizierung:

Jetzt ist HDL/Verilog¹ verfügbar. Der nächste Schritt ist die Verifizierung. Die Verifikation kann formal oder funktional sein. Eine fortgeschrittene formale Verifikation kann mit Cadence JasperGold durchgeführt werden, das im Moment nicht verfügbar ist. Die grundlegende Verifikation kann mit XCELIUM durchgeführt werden.

Vor-Synthese-Verifizierung:

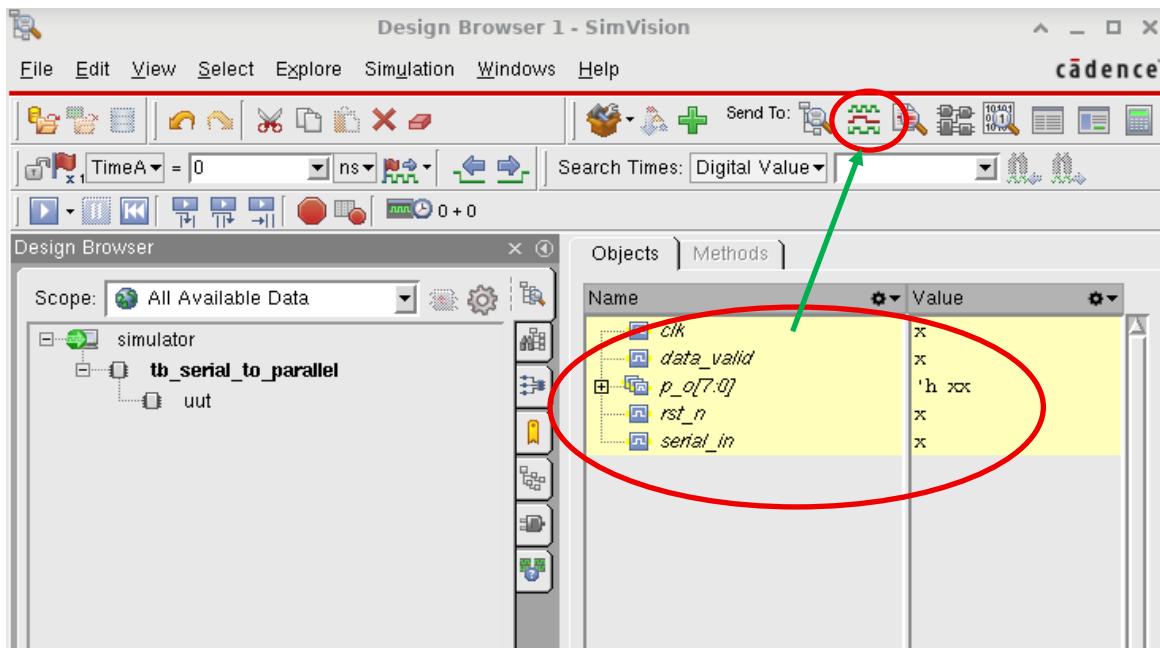
Für die Verifikation wird XCELIUM verwendet. Unter der Voraussetzung, dass die Verilog-Datei und die Testbench² verfügbar sind, kann der folgende Befehl die SimVision-Kontrollfelder aufrufen:

```
>> xrun SIPO1_tb.v -access +rwc -64bit -gui &
```

Hier ist **SIPO1_tb.v** die Testbench-Datei, die die Datei **SIPO_Multi-Clock_Domain.v** instanziiert und enthält. Sie kann auch folgendermaßen geschrieben werden, wenn die Datei **SIPO_Multi-Clock_Domain.v** nicht in der Testbench enthalten ist, sondern instanziiert wird:

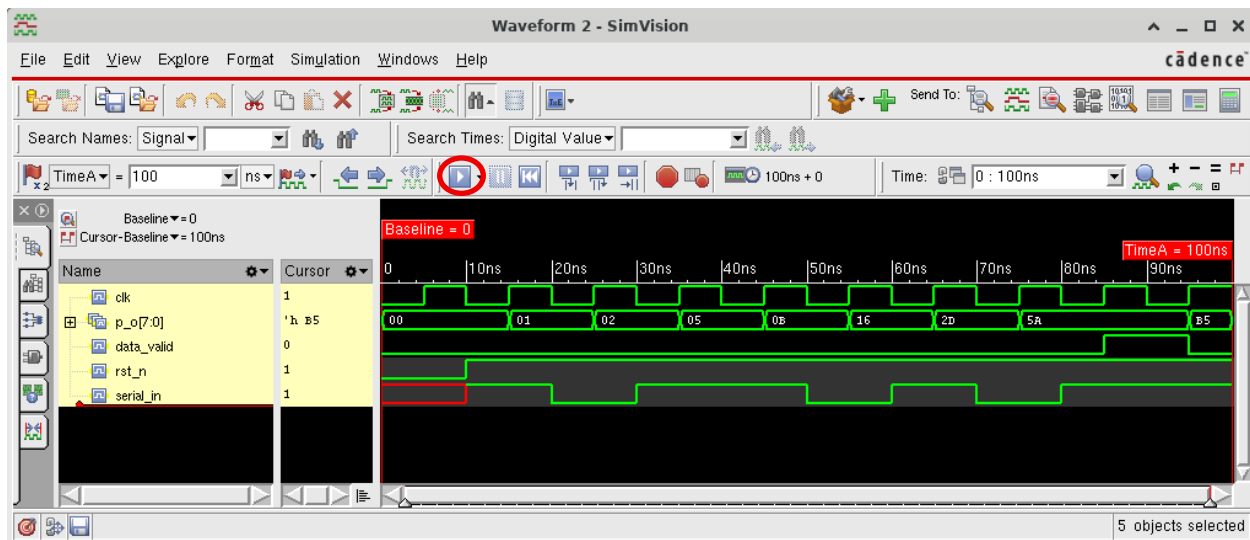
```
>> xrun SIPO1_tb.v SIPO_Multi-Clock_Domain.v -access +rwc -64bit -gui &
```

Die Angabe „-64bit“ ist optional. Kann notwendig sein, wenn der Entwurf zu groß ist.



¹ https://github.com/baruaeee/DD_Lab_exercise/blob/master/SIPO/Verification/Pre-Synthesis/SIPO_Multi-Clock_Domain.v

² https://github.com/baruaeee/DD_Lab_exercise/blob/master/SIPO/Verification/Pre-Synthesis/SIPO1_tb.v



Synthesis

Für Synthesis, zuerst bracht man die SDC Datai und Liberty Datai von PDK. Eine grundlegende SDC³ Datai ist wie folgendes:

SDC Version

set_sdc_version 2

set_units -time 1ps

Taktperiode definieren

create_clock -period 400 -name clk -waveform {0.0 200} [get_ports clk]

Lastkapazität für die Ausgangsanschlüsse (zu Demonstrationszwecken wird eine Last von 0,5 angenommen)

set_load 0.5 [get_ports p_o]

Einstellung der maximalen Übergangszeit für die Ausgangsports

set_max_transition 0.2 [get_ports p_o]

Einstellen der Taktübergangszeit

set_clock_transition 0.03 [get_clocks clk]

Einstellen der Taktunsicherheit für Setup und Hold

set_clock_uncertainty -setup 0.06 [get_clocks clk]

set_clock_uncertainty -hold 0.03 [get_clocks clk]

³ https://github.com/baruaeeee/DD_Lab_exercise/blob/master/SIPO/Synthesis/IHP_Open-PDK/SDC/SIPO.sdc

Einstellen der Eingangsverzögerung für die Eingangsanschlüsse (unter der Annahme einer maximalen Eingangsverzögerung von 1 ps)

set_input_delay -max 1.0 -clock clk [get_ports {serial_in rst_n}]

Einstellen der Ausgangsverzögerung für die Ausgangsanschlüsse (unter der Annahme einer maximalen Ausgangsverzögerung von 2 ps und einer minimalen von 1,5 ps)

set_output_delay -max 2.0 -clock clk [get_ports p_o]

set_output_delay -min 1.5 -clock clk [get_ports p_o]

Set false path for the reset signal to avoid timing issues

set_false_path -from [get_ports rst_n]

Liberty Datei kan in “https://github.com/IHP-GmbH/IHP-Open-PDK/tree/main/ihp-sg13g2/libs.ref/sg13g2_stdcell/lib” gefunden.

Synthesis ohne DFT:

Vorausgesetzt, die SDC, Liberty und HDL datai sind verfügbar, eine TCL Datai kan geschriben und das GENUS consol kann aufrufen. Die TCL Datai (ohne DFT) ist wie folgende:

Lädt die Standard Cell Library für die Synthese (IHP sg13g2 PDK, typische Corner, 1,8V, 25°C)

read_libs /home/bas33767/Desktop/IHP-Open-PDK/ihp-sg13g2/libs.ref/sg13g2_stdcell/lib/sg13g2_stdcell_typ_1p20V_25C.lib

Liest die Verilog HDL-Datei für den SIPO-Entwurf (Serial-In-Parallel-Out)

read_hdl ../../Verification/Pre-Synthesis/SIPO_Multi-Clock_Domain.v

Erarbeitet das Top-Level-Modul „serial_to_parallel“ aus der HDL

elaborate serial_to_parallel

Liest die Synopsys Design Constraints (SDC) Datei für Timing- und andere Constraints

read_sdc SDC/SIPO.sdc

Konfiguriert den hohen Aufwand für drei Synthesestufen: Generische Synthese (technologieunabhängig), Technologie-Mapping, Optimierung

set_db syn_generic_effort high

set_db syn_map_effort high

set_db syn_opt_effort high

Führt die drei Syntheseschritte nacheinander aus

syn_generic

syn_map

Schreiben der hdl-Datei nach dem Mapping für die Zwischen-DFT (optional)

write_hdl -lec > outputs/lec_src/SIPO2mapped_lec.v

LEC-Do-Datei für den Haupt-RTL-Entwurf (Golden Design) im Vergleich zum Zwischenentwurf (Mapped Design)

z.B. write_do_lec -golden_design [golden/file/path==>optional if the golden is the main design] -revised_design [revised_design/file/path] -logfile <log/file/path] > [do/file/path]

*write_do_lec -revised_design output/lec_src/SIPO2mapped_lec.v -logfile
outputs/lec_src/SIPO2mapped_lec.log > outputs/lec_src/SIPO2mapped_lec.do
syn_opt*

#Berichte

report_timing > reports/report_timing.rpt

report_power > reports/report_power.rpt

report_area > reports/report_area.rpt

report_qor > reports/report_qor.rpt

#Ergebnisse

write_hdl > outputs/SIPO_netlist.v

write_sdc > outputs/post_synth_SIPO.sdc

*write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -setuphold split >
outputs/delays.sdf*

write_design -base_name innovus_src/SIPO

LEC do-Datei für Zwischenmuster (mapped) und endgültiges Design

*write_do_lec -golden_design output/lec_src/SIPO2mapped_lec.v -revised_design
outputs/SIPO_netlist.v -logfile outputs/lec_src/mapped2final_lec.log >
outputs/lec_src/mapped2final_lec.do*

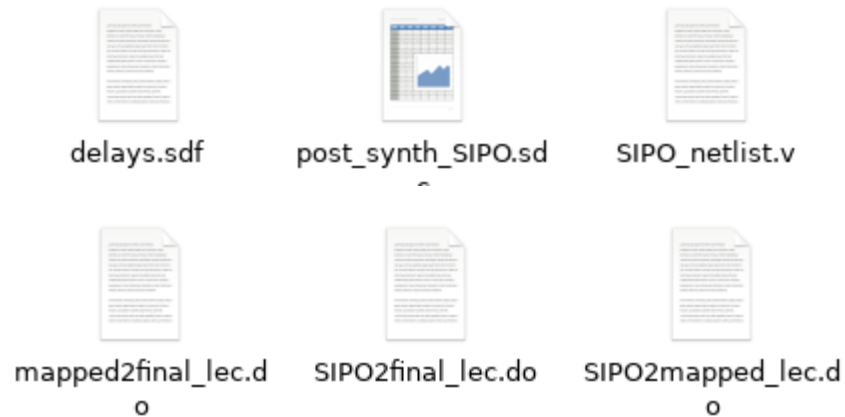
LEC do-Datei für den Haupt-RTL-Entwurf (goldener Entwurf) im Vergleich zum endgültigen Design

*write_do_lec -revised_design outputs/SIPO_netlist.v -logfile
outputs/lec_src/SIPO2final_lec.log > outputs/lec_src/SIPO2final_lec.do*

Das GENUS Consol kann aufrufen mit dem folgenden (und synth.tcl Datai) Befehl aufgerufen werden:

genus -f synth.tcl

Es werden alle erforderlichen Daten für den nächsten Schritt erzeugt.



Synthesis mit DFT

Die Synthese mit DFT erfordert zusätzliche Befehle, die in der neuen TCL-Datei enthalten sind.

Lädt die Standardzellenbibliothek für die Synthese

*read_libs /home/bas33767/Desktop/IHP-Open-PDK/ihp-
sg13g2/libs.ref/sg13g2_stdcell/lib/sg13g2_stdcell_typ_1p20V_25C.lib*

Liest die Verilog HDL-Datei für den SIPO-Entwurf (Serial-In-Parallel-Out)

read_hdl ../../Verification/Pre-Synthesis/SIPO_Multi-Clock_Domain.v

Erarbeitet das Top-Level-Modul „serial_to_parallel“ aus der HDL

elaborate serial_to_parallel

Liest die Synopsys Design Constraints (SDC) Datei für Timing- und andere Constraints

read_sdc SDC/SIPO.sdc

*# Konfiguriert den hohen Aufwand für drei Synthesestufen: Generische Synthese
(technologieunabhängig), Technologie-Mapping, Optimierung*

set_db syn_generic_effort high

set_db syn_map_effort high

set_db syn_opt_effort high

Führt die drei Syntheseschritte nacheinander aus

syn_generic

DFT

DFT-Scan FF-Stil für Scan-Ersatz

set_db dft_scan_style muxed_scan

Präfix zur Bezeichnung der DFT-Logik

set_db dft_prefix DFT_

Definieren Sie die Prüfsignale

define_dft shift_enable -name {scan_en} -active {high} -create_port {scan_en}

DFT-Regeln prüfen (mehrfach)

check_dft_rules

check_dft_rules

check_dft_rules

*#Konfigurieren der Anzahl der Scan-Ketten, die erforderlich sind, um alle FFs zu verbinden
(1 Scan-Kette verwendet, mehrere Ketten können getestet werden, um zu prüfen, ob die
Taktgeschwindigkeit verbessert werden kann)*

set_db current_design .dft_min_number_of_scan_chains 1

Angabe der Eingangs- und Ausgangsports der scan chain

define_dft scan_chain -name top_chain -sdi scan_in -sdo scan_out -create_ports

syn_map

syn_opt

*# Scan-Chains mit dem RC-Befehl connect_scan_chains verbinden. Dies schließt alle
ursprünglichen FFs ein, die auf Scan-Flops abgebildet wurden.*

connect_scan_chains -auto_create_chains

#Berichte

report_dft_chains > reports/report_dft_chains

report_timing > reports/report_timing.rpt

report_power > reports/report_power.rpt

report_area > reports/report_area.rpt

report_qor > reports/report_qor.rpt

#Ergebnisse

write_hdl > outputs/SIPO_netlist.v

write_sdc > outputs/post_synth_SIPO.sdc

write_sdf -timescale ns -nonegchecks -recrემ split -edges check_edge -setuphold split >
outputs/delays.sdf

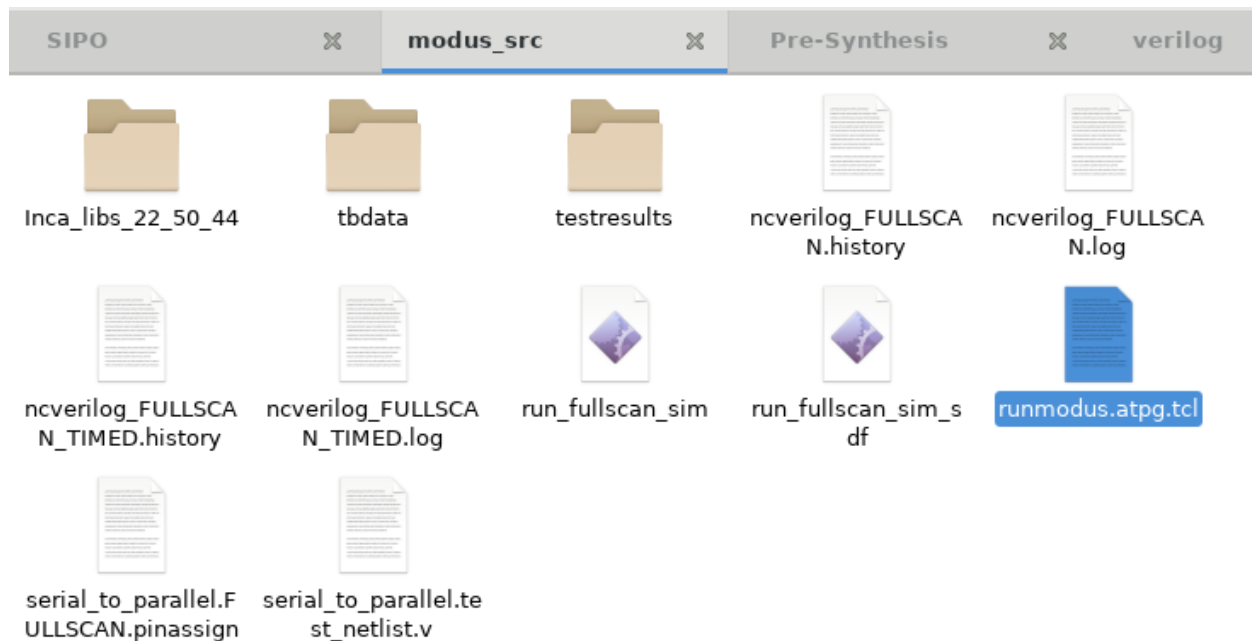
write_design -base_name innovus_src/SIPO

```
write_dft_atpg -library /home/bas33767/Desktop/IHP-Open-PDK/ihp-
sg13g2/libs.ref/sg13g2_stdcell/verilog/sg13g2_stdcell.v -directory outputs/modus_src
```

LEC do file for main RTL (golden) design design vs Final design

```
write_do_lec -revised_design outputs/SIPO_netlist.v -logfile
outputs/lec_src/SIPO2final_lec.log > outputs/lec_src/SIPO2final_lec.do
```

Es wird alle notwendigen Dateien für LEC zusammen mit den grundlegenden ATPG-Dateien für MODUS generieren.



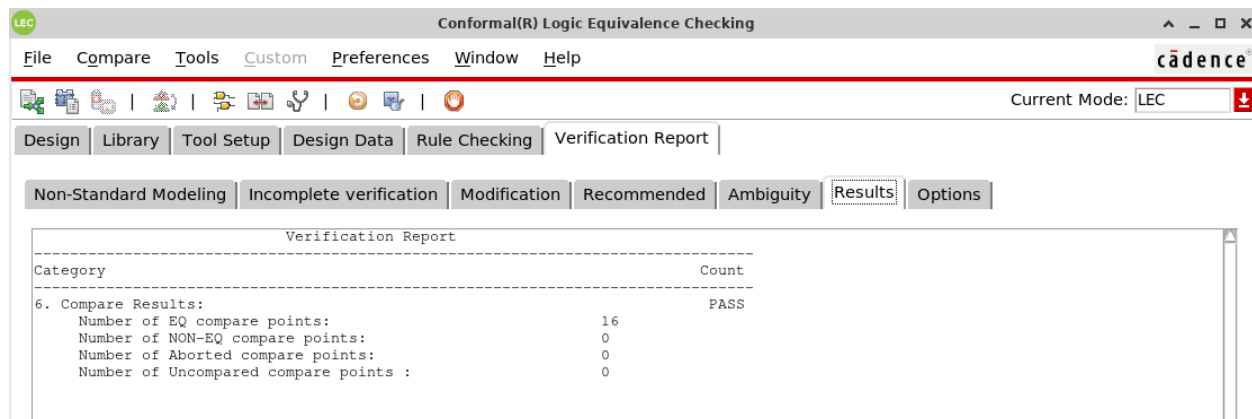
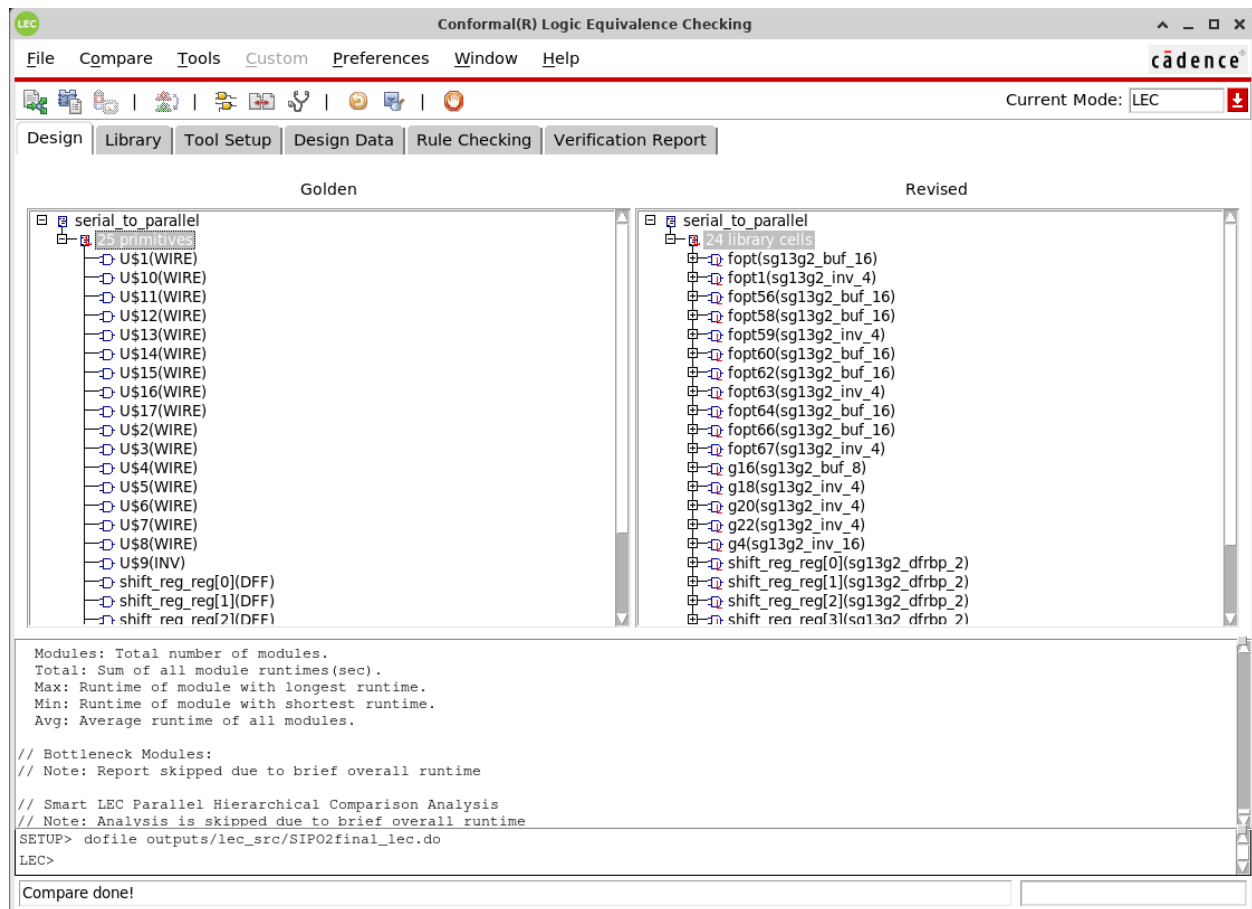
Post-Synthesis Verification

LEC:

Die Logic Equivalent Check (LEC) ist Teil der grundlegenden formalen Verifikation. Während der Synthesen werden einige 'do'-Daten erzeugt. Diese do-Daten kann man für LEC direkt aus der GENUS-Konsole nutzen.

```
>> lec -xl -dofile outputs/lec_src/SIPO2final_lec.do
```

Standardmäßig wird das tcl-Skript nach der Ausführung die COFORMAL-Konsole verlassen. Um das Ergebnis in der GUI zu sehen, muss der Befehl „exit“ in der allerletzten Zeile des Skripts kommentiert oder gelöscht werden.



Simulation auf Gate-Level:

Dies ist eine grundlegende Funktionsprüfung. In diesem Schritt muss die Testbench mit den SDF-Daten annotiert werden.

```

14 module tb_serial_to_parallel;
15
16     reg clk;           // Clock signal
17     reg rst_n;         // Reset signal (active low)
18     reg serial_in;     // Serial input
19     wire [7:0] p_o;    // Parallel output (8-bit)
20     wire data_valid;   // Signal indicating valid data
21
22     // Instantiate the serial_to_parallel module
23     serial_to_parallel uut (
24         .clk(clk),
25         .rst_n(rst_n),
26         .serial_in(serial_in),
27         .p_o(p_o)
28     //     .data_valid(data_valid)
29     );
30
31     // Post-Synthesis SDF Annotation starts
32
33     initial
34         begin
35             $sdf_annotate ("outputs/delays.sdf", uut);
36         end
37
38
39     // Post-Synthesis SDF Annotation ends
40
41     // Clock generation
42     initial clk = 0;
43     always #5 clk = ~clk; // Generate a clock with a period of 10 time units
44

```

>> xrun SIPO_gate-level_tb.v /home/bas33767/Desktop/IHP-Open-PDK/ihp-sg13g2/libs.ref/sg13g2_stdcell/verilog/sg13g2_stdcell.v -access +rwc -64bit -gui &

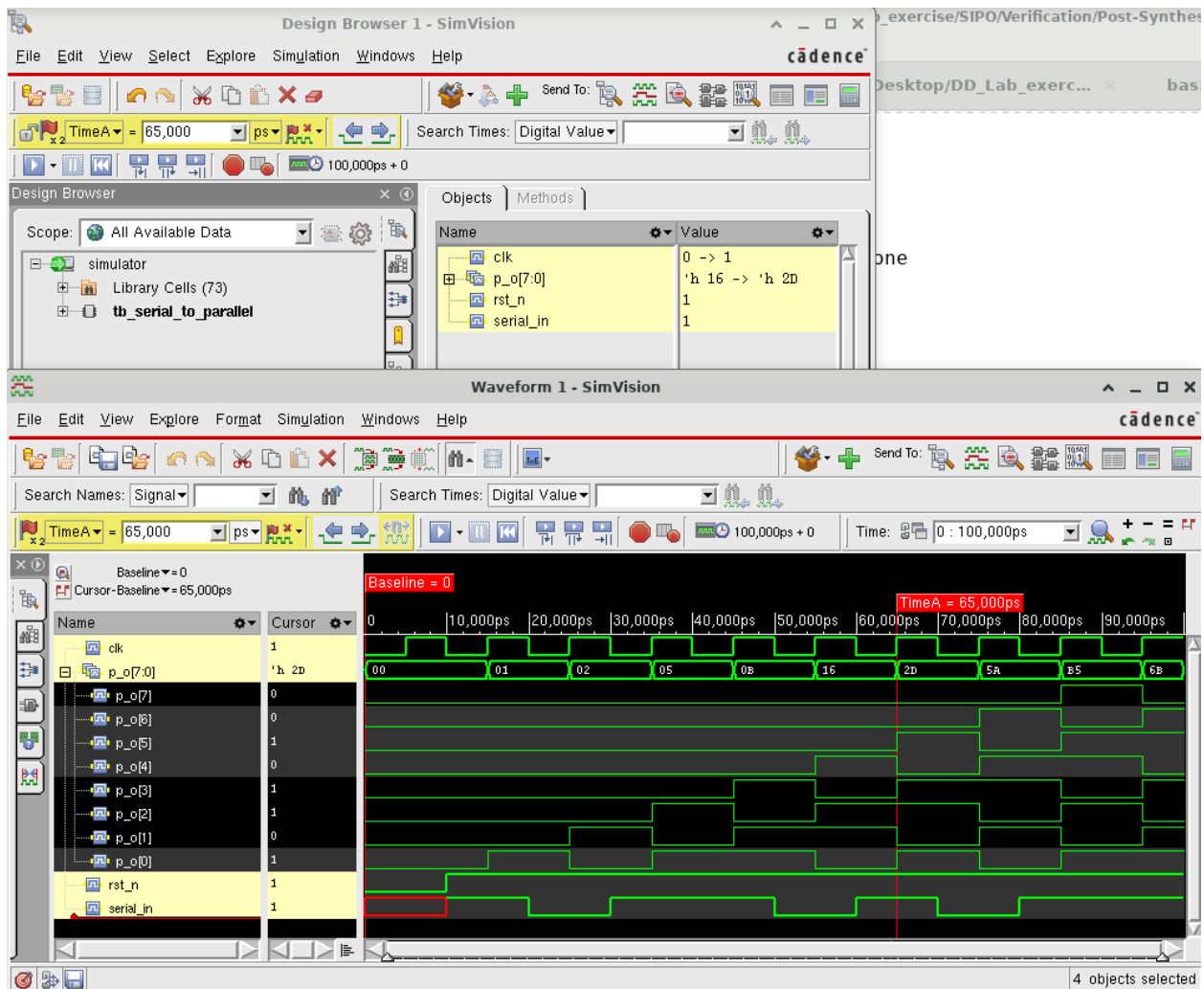
Wenn der PDK-Verilog-Daten in der Testbench enthalten ist:

```

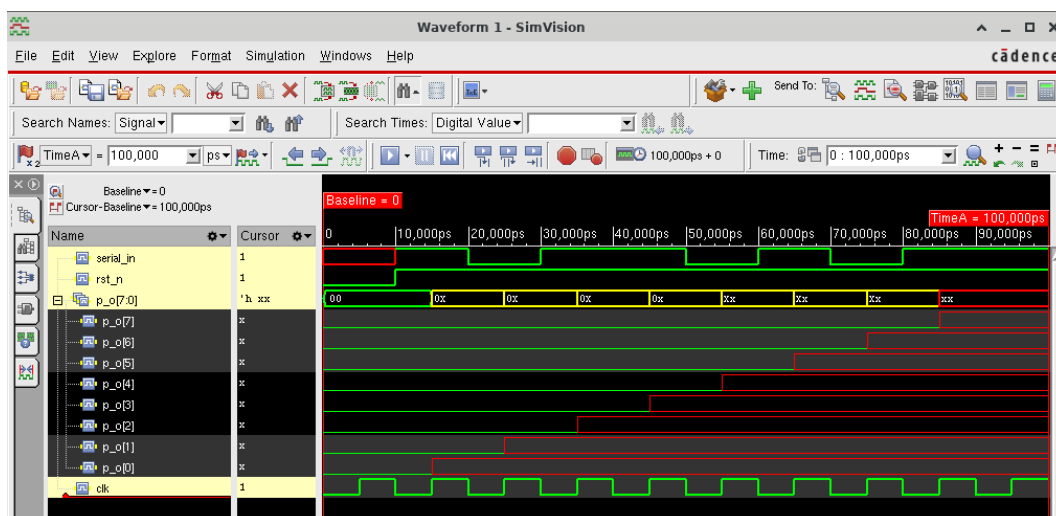
4
5 `include "outputs/SIPO_netlist.v"
6 `include "/home/bas33767/Desktop/IHP-Open-PDK/ihp-sg13g2/libs.ref/sg13g2_stdcell/verilog/sg13g2_stdcell.v"
7

```

>> xrun SIPO_gate-level_tb.v -access +rwc -64bit -gui &



Die Netzliste hat Scan-Chain FF hinzugefügt, der scan_en Pin muss low sein, um die funktionale Verifikation mit XCELIUM zu ermöglichen. Andernfalls wird die resultierende Wellenform inkonsistent sein, wie in der Abbildung unten gezeigt:



Um dies zu vermeiden, muss der Testbench modifiziert werden (scan_n-Port hinzufügen und ihn permanent niedrig halten).

```
.....
initial clk = 0;
always #5 clk = ~clk; // Generate a clock with a period of 10 time units

// Test sequence
initial begin
    rst_n = 0; // Assert reset
    scan_en = 0;
    #10 rst_n = 1; // Deassert reset after 10 time units

    // Send 8 bits serially
    serial_in = 1'b1; #10
    serial_in = 1'b0; #10
end
```

DFT:

Modus kann auch über die Genus-Konsole ausgeführt werden.

```
>> modus
```

```
>> source outputs/modus_src/runmodus.atpg.tcl
```

Das Skript „runmodus.atpg.tcl“ führt (wie in „SIPO2final_lec.do“) dazu, dass die Modus-Konsole beendet wird. Um dies zu vermeiden, muss die Zeile „exit“ gelöscht oder auskommentiert werden.

```
>> gui_open
```

The screenshot displays the Cadence Modus DFT Software Solution interface. The main window is titled "Cadence Modus DFT Software Solution: /home/bas33767/Desktop/DD_Lab_exercise/SIPO/Synthesis/IHP_Open-PDK_DFT/outputs/modus_src". The interface includes a "Tasks" panel on the left, a "Log" panel in the center, and a "Text" panel on the right.

The "Tasks" panel shows a list of tasks, including "build_model", "build_testmode", "report_test_structures", "verify_test_structures", "build_faultmodel", "create_logic_tests", "write_toggle_gram", "write_vectors", and "commit_tests". The "commit_tests" task is currently selected.

The "Log" panel displays the "Log For Commit Tests" summary, showing 14 numbered messages. The messages include "Error (0)", "Severe Warning (0)", "Warning (0)", and "Informational (14)".

The "Text" panel shows the output of the "commit_tests" task, including a table of testmode and global statistics after commit_tests.

ATCov		Total	Tested	Testmode Faults	Redunda
Testmode	Global			Possibly	
Total Static	0.00 0.00	220	0	0	
Collapsed Static	0.00 0.00	130	0	0	
PI Static	0.00 0.00	10	0	0	
PO Static	0.00 0.00	18	0	0	
Total Dynamic		0			
Parametric					
IDDq	0.00 0.00	220	0		
Path	0.00 0.00	0	0		

The "Text" panel also shows the output of the "commit_tests" task, including a table of testmode and global statistics after commit_tests.

ATCov		Total	Tested	Testmode Faults	Redunda
Testmode	Global			Possibly	
Total Static	100.00 100.00	220	206	0	
Collapsed Static	100.00 100.00	130	116	0	
PI Static	100.00 100.00	10	10	0	
PO Static	100.00 100.00	18	18	0	
Total Dynamic		0			
Parametric					
IDDq	0.00 0.00	220	0		
Path	0.00 0.00	0	0		

Es gibt zwei zusätzliche Skripte zur Ausführung von XCELIUM, um die DFTs zu testen. Das eine ist ohne SDF-Datei, das andere mit SDF. Das Skript mit SDF-Datei muss modifiziert werden (ändern Sie den Pfad von DEFAULT SDF), andernfalls wird es fehlschlagen. Diese Skripte können direkt von genus consol abgerufen werden.

>> source outputs/modus_src/run_fullscan_sim

```
INFO (TVE-210): Results for vector file: outputs/modus_src/testresults/verilog/VER.FULLSCAN.serial_to_parallel_1.ts2.verilog
      Number of Cycles:          29
      Number of Tests:           4
      - Passed Tests:            4
      - Failed Tests:            0
      Number of Compares:        59
      - Good Compares:           59
      - Miscompares:             0
      Time:                      2800000.00 ps

INFO (TVE-209): Cumulative Results:
      Number of Files Simulated:  2
      Total Number of Cycles:     35
      Total Number of Tests:      5
      - Total Passed Tests:       5
      - Total Failed Tests:       0
      Total Number of Compares:   67
      - Total Good Compares:      67
      - Total Miscompares:        0

Simulation complete via $finish(1) at time 2800 NS + 0
./outputs/modus_src/testresults/verilog/VER.FULLSCAN.serial_to_parallel_atpg.mainsim.v:228      $finish;
xcelium> exit
xmsim: Memory Usage - Final: 55.5M, Peak: 55.5M, Peak virtual: 116.5M
xmsim: CPU Usage - 0.1s system + 0.0s user = 0.1s total (1.1s, 8.8% cpu)
T00L:  xrun(64)      24.03-s004: Exiting on May 29, 2025 at 01:32:18 CEST (total: 00:00:02)
@genus:root: 5> █
```